Hands-on Activity 11.1 | More Components and Styling	
Course Code: CPE 025B	**Program:** Computer Engineering
Course Title: Software Design 2	**Date Performed: 03/29/26**
Section: CPE32S1	**Date Submitted: 03/29/26**
Name: Edson Ray San Juan	**Instructor: Engr. Neil Matira**

1. Objective(s)

This activity aims to finalize the simple React Native application developed in the previous activity by allowing students to utilize more components and implement further styling techniques.

2. Intended Learning Outcomes (ILOs)

After this module, the student should be able to:

- Utilize React Native components to make scrollviews and optimized lists; and
- Decompose the app component into multiple custom components.

3. Discussion

This laboratory does not come with a separate discussion.

4. Materials and Equipment

To properly perform this activity, the student must have:

- Node.js LTS
- Expo App
- Android Emulator/Simulator (or any iOS/Android mobile phone)

5. Procedure

Sample Goal List Application from the Previous Activity

Before we begin with this section, include a screenshot of your goal list application after you've

finished the application so far in the previous activity.

Making Scrollable Content with ScrollView

1. In your goal list application, keep adding goals until the screen can no longer hold the elements. Similar to the image above.
2. Import *ScrollView* from React Native and change the view component containing your list of goals into
   <ScrollView></ScrollView>. Note that style must be retained. Save and show the output.
3. What can you say about how the ScrollView looks now with your goals?
4. Now, create another view component such that your ScrollView becomes encapsulated by the <View></View> component. The style from the ScrollView component must be applied to this new View component instead. Did anything change? What do you think is the benefit of having the ScrollView component contained by the View component?
5. Create a new style object called *goalListContainer* and assign it to the View component containing your ScrollView. Manipulate it by using props like height and flex. Observe what happens to the scrollable content; add screenshots for the tests you perform.

Optimizing Lists with FlatList

A common problem with using ScrollView is that even if items are not yet viewable, they are still rendered. ScrollView renders all its items regardless of the number. This becomes a performance issue when you have tens of thousands of items in the item – and they are rendered even if they're not viewable. So, for situations like that, we do not want to use ScrollView. We will use another component called *FlatList* to solve this problem.

Optimizing Lists with FlatList

A common problem with using ScrollView is that even if items are not yet viewable, they are still rendered. ScrollView renders all its items regardless of the number. This becomes a performance issue when you have tens of thousands of items in the item – and they are rendered even if they're not viewable. So, for situations like that, we do not want to use ScrollView. We will use another component called *FlatList* to solve this problem.

1. Import FlatList and change ScrollView in your function App to FlatList.
2. Remove the mapping of the course goals that was done previously. This task will now be passed to FlatList. Your code must look similar to below.

```
<FlatList>
  <Text style={styles.goalItems} key={goal}>{goal}</Text>
</FlatList>
```

3. In order to pass this task to FlatList, we must use *data* prop and pass our list.
4. Remove the text component and turn FlatList into a self-closing component.
5. Add another required prop called *requiredItem*. This prop requires a function to tell it how to render the items in the given data. In this case, we will add the code we removed from earlier as the content of an arrow function.

In the example below, the arrow function passed to the requiredItem prop is the item function, which returns the Text component from earlier. Your code must look similar to the example below.

```
<View style={styles.goalListContainer}>
  <FlatList data={courseGoals} renderItem={(item) => {
    return(
      <Text style={styles.goalItems} key={goal}>{goal}</Text>
    )
  }} />
</View>
```

6. The *item* passed here is actually an object generated by FlatList internally that is wrapped around the individual data items. We will rename it to *itemData* to fully reflect this function. This object has its own properties, some of which we will get to see. We can track the index using itemData.index, but the prop we need here is *itemData.item* which gives us the individual items in the data.

   So, change the goal to *itemData.item* and remove the key prop from the view, since we are no longer mapping it manually. Show the output.
7. Since we are still dealing with lists, keys are still important and nowhere in our code a key exists. To solve this, we will turn our data values from primitive values like strings into objects that have a key property.

   Return to your addGoalHandler() function and change enteredGoalText by wrapping it into an object and add a text property and a key property. Your code should look similar to

```
function addGoalHandler() {
  setCourseGoals((currentCourseGoals) =>
  [...currentCourseGoals,
  {text: enteredGoalText, key: }]);
```

   Once this has been added, you must define your key value. For the purpose of this procedure, we will use *Math.random().toString()*. FlatList will now automatically look for the keys associated with the values.
8. Now, since we're dealing with objects with keys, we have to ensure that we are able to access the text property of the items. Access the text property for the *itemData.item* below.

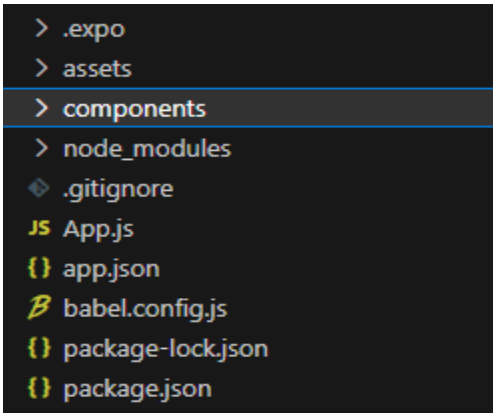
9. Reload the app and add multiple items in your list with the same value. What can you observe?

<u>Splitting Components into Smaller Components</u>

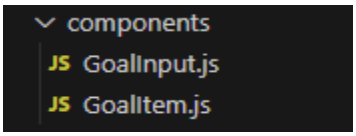
Notice that as we develop this basic application further, our code keeps getting larger and our components also keep getting larger. The same happens with applications developed in

React Native.

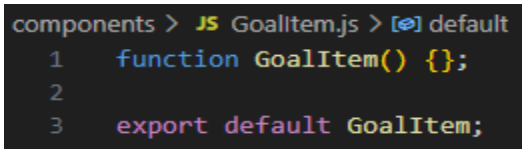
1. Create a new folder called *components*. This is where we will store our components.



2. Create two files for the components, this will contain the actual JSX code and functionality related to our components. Name the files *GoalInput.js* and *GoalItem.js*.



3. In the GoalItem.js, create a function goal item and export as default.



4. Within your GoalItem component, you want it to return the JSX code responsible for outputting a single goal item. Not the list of items, but a single item.

   In our App.js, this is the section of the code responsible for that.

```
<View style={styles.goalContainer}>
  <View style={styles.headerContainer}>
  <Text style={styles.headerText}>LIST OF GOALS</Text>
  </View>
  <View style={styles.goalListContainer}>
    <FlatList data={courseGoals} renderItem={(itemData) => {
      return(
        <View style={styles.goalItems}>
        <Text style={styles.goalText}>{itemData.item.text}</Text>
        </View>
      )
    }}
    />
  </View>
</View>
```

Cut it, and return it as your item in GoalItem component.

```
components > JS GoalItem.js > default
1  function GoalItem() {
2      return(
3          <View style={styles.goalItems}>
4              <Text style={styles.goalText}>{itemData.item.text}</Text>
5          </View>
6      );
7  }
```

5. All the styles being referenced also have to be brought over. Create a stylesheet object in GoalItem.js and include all the associated styles. Cut them from your App.js. *Hint: It is good practice to keep your styles close to your JSX code.*

6. Clean up your code by removing unused imports. Import your GoalItem.js.

```
import GoalItem from './components/GoalItem';
```

7. Now, down in your JSX code, where the code for the return item used to be, return a self-closing GoalItem component instead. Show your code so far.
8. Is this easier than the previous method performed? What benefits do you see to applying this technique?

Note: Running the code as it is right now will return an error. Finish the procedure first before trying to run the code.

<u>Utilizing Props</u>

If you did not heed the advice from the note and ran your code anyway, you would have probably gotten an error. This error can be solved now with the use of *props*.

```
function GoalItem() {
    return(
        <View style={styles.goalItems}>
            <Text style={styles.goalText}>{itemData.item.text}</Text>
        </View>
    );
}
```

Looking at the snippet above, because we cut the code from App.js to our custom component; there is no way yet for the component to know what this itemData.item.text is, unlike beforehand. To solve this, we'll use props:

1. Add *props* as a keyword to your GoalItem() function and inside the Text component's display, where itemData.item.text is, replace with *props.text*.
2. Go back to your App.js, we will set a text prop to the GoalItem component being returned within the FlatList, this must be *text={itemData.item.text}*. Save, run and show your output. Try adding a goal. Is your output similar to the image below? Why do you think that is?

3. Solve by importing View and Text components. Save the code and try inputting a goal in the app. What output did you get and why do you think so?

<u>Working on the "Goal Input" Component</u>

We will basically apply the same steps as we did earlier for the GoalItem component but this time for our other custom component, GoalInput.

1. Create the functional component for GoalInput. Include *export default GoalInput;* at the end of this function as well.

```
components > JS GoalInput.js > default
function GoalInput() {
    return(

    );
}
```

2. Identify from the App.js the JSX responsible for the text input of our goal. In this figure, it is the highlighted portion, we are including the view component for this because that is the container of our textInput component as well.

```
return (
  <View style={styles.appContainer}>
    <View style={styles.inputContainer}>
      <TextInput placeholder='Your course goals!'
      style={styles.textInput}
      onChangeText={textInputHandler}
      />
      <Button title='Add Goal' onPress={addGoalHandler} color={'#3f7579'}/>
    </View>
```

Your GoalInput.js should look similar to the figure below.

```
components > JS GoalInput.js > default
1   function GoalInput() {
2       return(
3           <View style={styles.inputContainer}>
4               <TextInput placeholder='Your course goals!'
5               style={styles.textInput}
6               onChangeText={textInputHandler}
7               />
8               <Button title='Add Goal' onPress={addGoalHandler} color={'#3f7579'
9           </View>
10      );
11  }
```

3. Addressing the same issues as encountered earlier, make sure to import the necessary components from react native by adding the following line of code at the top of your custom component.

```
components > JS GoalInput.js > styles > inputContainer
1   import { View, TextInput, Button, StyleSheet } from 'react-native';
```

Additionally, you want to include all the necessary styles to be close to your JSX file.

```
const styles = StyleSheet.create({

})
```

At this point, you may already realize some things will go wrong with the finished component file. What are the problems you expect and why?

4. Similar to our earlier solution, one problem can be solved instantly through the utilization of *props*. Include a prop to your functional component like in the figure.

```
function GoalInput(props)
```

This will be used to talk to the parent component and activate upon press. Modify *onPress={addGoalHandler}* so that it utilizes the prop like in the figure below.

```
<Button title='Add Goal' onPress={props.onAddGoal} color={'#3f7579'}/>
```

5. In your App component, input the custom component GoalInput.js. Then use the component in the same place where your previous code was in the JSX. Show your code screenshot here.

6. Now, add the prop that we created in GoalInput component. Remember that *onPress* we want the button to handle the textinput and add it to the list. This means that, we just have to replicate the passing of a function done previously. What function should be passed here?

7. A new issue arises now: how do we handle the user input? Remember that *const [enteredGoalText, setEnteredText] = useState('');* is the useState we're using to handle the user input. Cut this from your App component and wrap it in your GoalInput function component. Your code should now be similar to the figure below.

   Do not forget to import the useState.

```
import { useState } from 'react';
import { View, TextInput, Button, StyleSheet } from 'react-native';

function GoalInput(props) {
const [enteredGoalText, setEnteredText] = useState('');

    return(
        <View style={styles.inputContainer}>
            <TextInput placeholder='Your course goals!'
            style={styles.textInput}
            onChangeText={textInputHandler}
            />
            <Button title='Add Goal' onPress={props.onAddGoal} color
        </View>
    );
}
```

You must also obtain the function meant to update the enteredGoalText.

```
function GoalInput(props) {
    const [enteredGoalText, setEnteredText] = useState('');

    function textInputHandler(enteredText) {
        setEnteredText(enteredText);
    };
```

These configurations make it so that the textInputHandler in your JSX file of GoalInput works

8. However, enteredGoalText is also still needed as the data for the list to be displayed in your app component. But we moved the useState for enteredGoalText to a different component. So, we need to find a way to make this work.

```
export default function App() {

const [courseGoals, setCourseGoals] = useState([]);

  function addGoalHandler() {
    setCourseGoals((currentCourseGoals) =>
    [...currentCourseGoals,
    { text: enteredGoalText, key: Math.random().toString() },]);
  };
```

A simple solution is to add *enteredGoalText* as the parameter for addGoalHandler(), refer to the image below.

```
function addGoalHandler(enteredGoalText)
```

But now, no text is provided yet.

9. In your GoalInput.js, create a new function called *addGoalHandler*. Yes, the same name! Name clashing will not occur here because we're in different components. In this new function, execute props.onAddGoal inside instead.

   Next, pass the new addGoalHandler to the onPress={} prop as a function object.

   Once onPress prop has the function as an object, we can pass enteredGoalText as

   the parameter of the prop. Your code should look similar to this figure.

```
function textInputHandler(enteredText) {
    setEnteredText(enteredText);
};

function addGoalHandler() {
    props.onAddGoal(enteredGoalText);
};

return(
    <View style={styles.inputContainer}>
        <TextInput placeholder='Your course goals!'
        style={styles.textInput}
        onChangeText={textInputHandler}
        />
        <Button title='Add Goal' onPress={addGoalHandler}
```

To clarify:

In your app component, you have defined appGoalHandler which takes an enterGoalText as a parameter. This same function is passed in your GoalInput component. This means that, if onAddGoal passes a value to addGoalHandler function, it becomes the value for enteredGoalText

```
function addGoalHandler(enteredGoalText) {
  setCourseGoals(currentCourseGoals) =>
  [...currentCourseGoals,
  { text: enteredGoalText, key: Math.random().toString() },]);
};


return (
  <View style={styles.appContainer}>
    <GoalInput onAddGoal={addGoalHandler} />
```

In your GoalInput component, you made a function with the same name (which can even be named anything else). This function will execute the prop to pass enteredGoalText to the function in the app component. This is standard React.

Lastly, we can add a value prop to contain enteredGoalText then use setEnteredText(‘’) as an empty text to reset the text input field within the addGoalHandler.

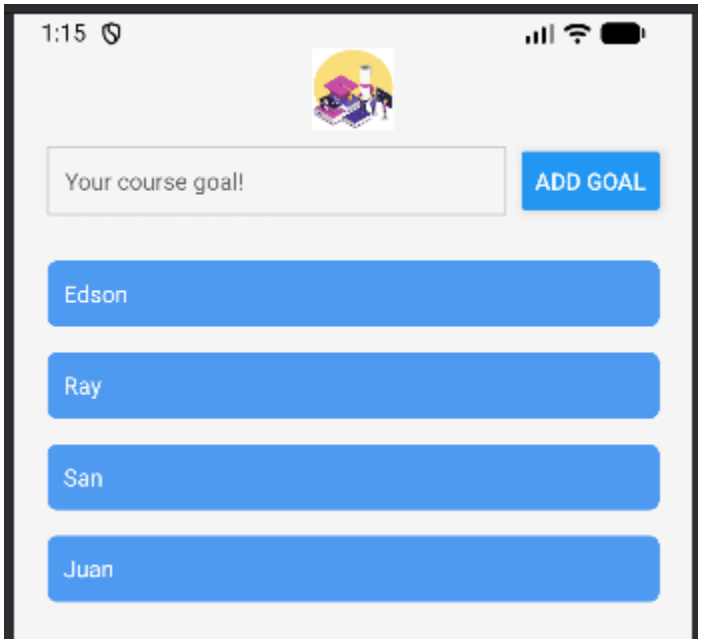
```
function addGoalHandler() {
    props.onAddGoal(enteredGoalText);
    setEnteredText('');
};
```

Your application should be working properly. If it isn't you must troubleshoot and document your process.

6. Output

<u>Making Scrollable Content with ScrollView</u>

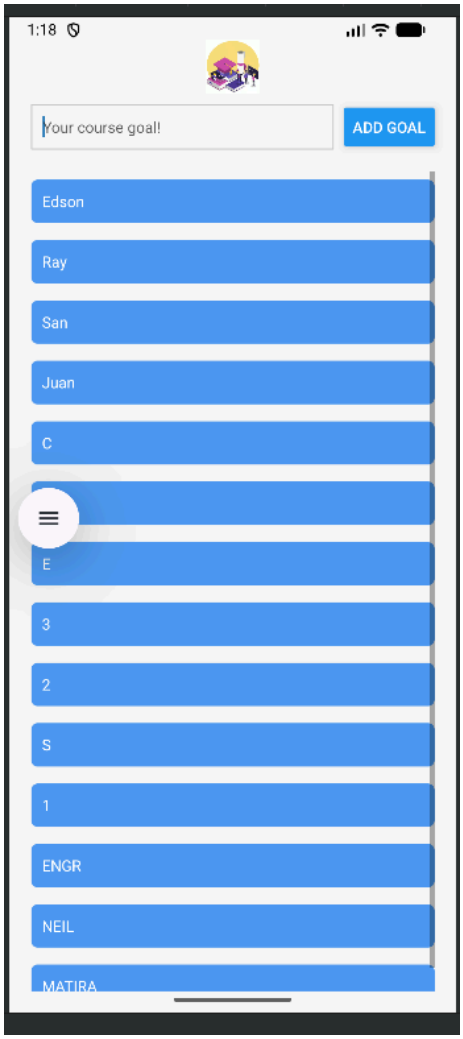
1. In your goal list application, keep adding goals until the screen can no longer hold the elements. Similar to the image above.



2. Import ScrollView from React Native and change the view component containing your list of goals into

```
<ScrollView style={styles.goalsContainer}>
  {courseGoals.map((goal) => (
    <Text key={goal} style={styles.goalItem}>{goal}</Text>
  ))}
</ScrollView>
```

3. <ScrollView></ScrollView>. Note that style must be retained. Save and show the output.



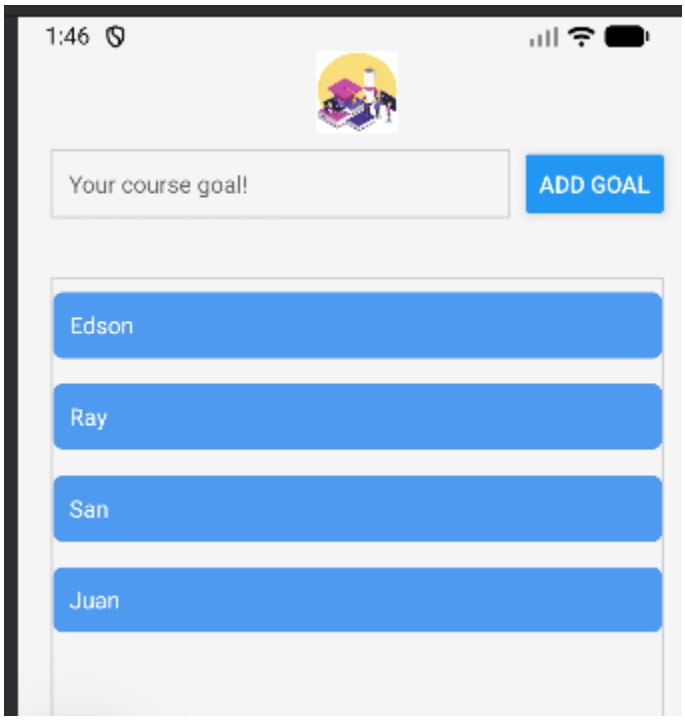
4. What can you say about how the ScrollView looks now with your goals?
   - Nothing major changes visually.
5. Now, create another view component such that your ScrollView becomes encapsulated by the <View></View> component. The style from the ScrollView component must be applied to this new View component instead. Did anything change? What do you think is the benefit of having the ScrollView component contained by the View component?

```
<View style={styles.goalListContainer}>
  <ScrollView>
    {courseGoals.map((goal) => (
      <Text key={goal} style={styles.goalItem}>{goal}</Text>
    ))}
  </ScrollView>
</View>
```

   - Encapsulating the ScrollView inside a View with goalListContainer doesn't change the visual output much, but it separates layout control from scrolling behavior
6. Create a new style object called goalListContainer and assign it to the View component containing your ScrollView. Manipulate it by using props like height and flex. Observe what

happens to the scrollable content; add screenshots for the tests you perform.

```
goalListContainer: {
  flex: 1,
  height: 300,
  marginTop: 16,
  borderWidth: 1,
  borderColor: '#ccc',
},
```



Optimizing Lists with FlatList

```
import { StyleSheet, Text, View, Button, TextInput, FlatList, Image } from 'react-native';
```

```
<View style={styles.goalListContainer}>
  <FlatList
    data={courseGoals}
    renderItem={(itemData) => (
      <Text style={styles.goalItem}>{itemData.item}</Text>
    )}
  />
</View>
```

- ScrollView issue: It renders all items, even those off-screen, which can slow performance for large lists.
- Replaced ScrollView with FlatList, stored goals as objects with { text, key }, and used renderItem to display them efficiently.

Optimizing Lists with FlatList

1.

```
import { StyleSheet, Text, View, Button, TextInput, FlatList, Image } from 'react-native';
```

- Then I replace ScrollView with FlatList

2. .MY OUTPUT

```
<View style={styles.goalListContainer}>
  <FlatList
    data={courseGoals}
    renderItem={(itemData) => (
      <Text style={styles.goalItem}>{itemData.item}</Text>
    )}
  />
</View>
```

3.

```
<View style={styles.goalListContainer}>
  <FlatList
    data={courseGoals}
    renderItem={(itemData) => (
      <Text style={styles.goalItem}>{itemData.item}</Text>
    )}
  />
</View>
```

4. SAME OUTPUT AS NUMBER 3
5. MY OUTPUT

```
<View style={styles.goalListContainer}>
  <FlatList
    data={courseGoals}
    renderItem={(itemData) => (
      <Text style={styles.goalItem}>{itemData.item}</Text>
    )}
  />
</View>
```

6. SAME OUTPUT AS NUMBER 5
7. MY OUTPUT

```
function addGoalHandler() {
  setCourseGoals((currentCourseGoals) => [
    ...currentCourseGoals,
    { text: enteredGoalText, key: Math.random().toString() },
  ]);
  setEnteredGoalText('');
}
```

8.

```
<View style={styles.goalListContainer}>
  <FlatList
    data={courseGoals}
    renderItem={(itemData) => (
      <Text style={styles.goalItem}>{itemData.item.text}</Text>
    )}
  />
</View>
```

9. Reload the app and add multiple items in your list with the same value. What can you observe?
   - Based on my observation, if I type let say Edson 2 times, FlatList will render them separately because each has a unique key. Therefore this avoids the warning in the previous in ScrollView

<u>Splitting Components into Smaller Components</u>

1. Create a new folder called *components*. This is where we will store our components.

```
> .expo
> assets
> components
> node_modules
  .gitignore
  App.js
  app.json
  babel.config.js
  package-lock.json
  package.json
```

2.

```
v components
  GoalInput.js
  GoalItem.js
```

3.

components > JS GoalItem.js > default

```
function GoalItem() {};

export default GoalItem;
```

4. MY OUTPUT

```
<View style={styles.goalListContainer}>
  <FlatList
    data={courseGoals}
    renderItem={(itemData) => (
      <Text style={styles.goalText}>{itemData.item.text}</Text>)}
  />
</View>
```

5.

```
const styles = StyleSheet.create({
  goalItems: {
    marginVertical: 8,
    padding: 10,
    backgroundColor: '#4e9af1',
    borderRadius: 6,
  },
  goalText: {
    color: 'white',
  },
});
```

6. MY OUTPUT

```
import GoalItem from './components/GoalItem';   // Import GoalItem
```

7. MY OUTPUT

```
<View style={styles.goalListContainer}>
  <FlatList
    data={courseGoals}
    renderItem={(itemData) => (
      <GoalItem text={itemData.item.text} />   /* Use GoalItem here */
    )}
  />
</View>
```

8. Is this easier than the previous method performed? What benefits do you see to applying this technique?
   - Is this easier than the previous method performed?

- Yes, because it is easier to locate, read and maintain, therefore the App.js stays clean

- What benefits do you see to applying this technique?
  - **Reusable** because GoalItem can be used anywhere else in your app. And **Organized** styles and logic are grouped with their components.

Utilizing Props

1.

```
function GoalItem(props) {
  return (
    <View style={styles.goalItems}>
      <Text style={styles.goalText}>{props.text}</Text>
    </View>
  );
}
```

2. Go back to your App.js, we will set a text prop to the GoalItem component being returned within the FlatList, this must be *text={itemData.item.text}*. Save, run and show your output. Try adding a goal. Is your output similar to the image below? Why do you think that is?

- I GOT THE IDENTICAL ERROR (BUT I CANT SCREENSHOT IT BECAUSE WHEN I TRY TO TYPE IN IT THE ERROR IS NOT DISPLAYING)

- MY OUTPUT

```
<View style={styles.goalListContainer}>
  <FlatList
    data={courseGoals}
    renderItem={(itemData) => (
      <GoalItem text={itemData.item.text} />
    )}
  />
</View>
```

3. Solve by importing View and Text components. Save the code and try inputting a goal in the app. What output did you get and why do you think so?

```
import { View, Text, StyleSheet } from 'react-native';
```

   - Based on my observation is that each output goal rendered correctly because the missing imports were added and the component could now display the text passed through props.

<u>Working on the "Goal Input" Component</u>

1. MY OUTPUT

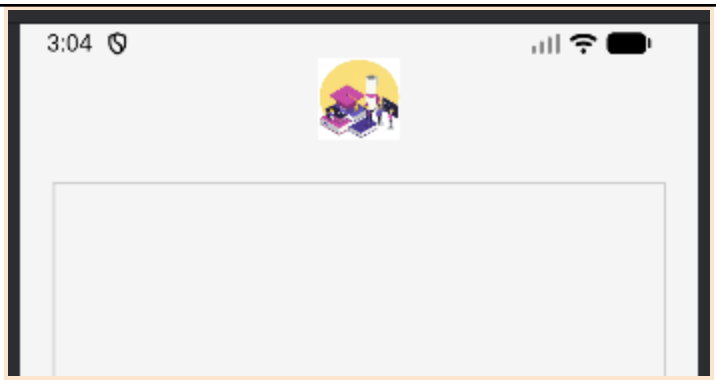
components > JS GoalInput.js > ...

```
function GoalInput() {
  return (
    // we'll add JSX here in the next step
  );
}

export default GoalInput;
```

2. DONE

3. At this point, you may already realize some things will go wrong with the finished component file. What are the problems you expect and why?
   - The problem is that textInputHandler and addGoalHandler are not defined inside GoalInput yet, so you'll get errors if you run this. THerefore the component doesn't know how to communicate with App.js yet, as seen in the output theres no input



4. DONE
5.

```
import GoalInput from './components/GoalInput';   // Import GoalInput
```

```
<GoalInput onAddGoal={addGoalHandler} />
```

6. Now, add the prop that we created in GoalInput component. Remember that *onPress* we want the button to handle the textinput and add it to the list. This means that, we just have to replicate the passing of a function done previously. What function should be passed here?
   - addGoalHandler function should pass, because this is the function responsible for updating the courseGoals state in App.js by adding a new goal to the list whenever the button inside GoalInput is pressed.

```
function addGoalHandler(enteredGoalText) {
  setCourseGoals((currentCourseGoals) => [
    ...currentCourseGoals,
    { text: enteredGoalText, key: Math.random().toString() },
  ]);
}
```

7. DONE
8. DONE
9. MY CODE

JavaScript

```
// GoalInput.js
import { useState } from 'react';
import { View, TextInput, Button, StyleSheet } from 'react-native';

function GoalInput(props) {
  const [enteredGoalText, setEnteredText] = useState('');
```

```
  function textInputHandler(enteredText) {
    setEnteredText(enteredText);
  }

  // Local handler that calls the parent function
  function addGoalHandler() {
    props.onAddGoal(enteredGoalText);
    setEnteredText(''); // reset input after adding
  }

  return (
    <View style={styles.inputContainer}>
      <TextInput
        placeholder="Your course goal!"
        style={styles.textInput}
        onChangeText={textInputHandler}
        value={enteredGoalText}
      />
      <Button title="ADD GOAL" onPress={addGoalHandler} color="#3f7579" />
    </View>
  );
}

const styles = StyleSheet.create({
  inputContainer: {
    flexDirection: 'row',
    justifyContent: 'space-between',
    alignItems: 'center',
    marginBottom: 20,
  },
  textInput: {
    borderWidth: 1,
    borderColor: '#cccccc',
    padding: 10,
    flex: 1,
    marginRight: 10,
    fontFamily: 'Arial',
  },
});

export default GoalInput;
```

JavaScript

```
//GoalItem.js
import { View, Text, StyleSheet } from 'react-native';

function GoalItem(props) {
  return (
```

```
    <View style={styles.goalItems}>
      <Text style={styles.goalText}>{props.text}</Text>
    </View>
  );
}

const styles = StyleSheet.create({
  goalItems: {
    marginVertical: 8,
    padding: 10,
    backgroundColor: '#4e9af1',
    borderRadius: 6,
  },
  goalText: {
    color: 'white',
  },
});

export default GoalItem;
```

JavaScript

```
// App.js
import { useState } from 'react';
import { StyleSheet, View, FlatList, Image } from 'react-native';
import GoalItem from './components/GoalItem';     // Import GoalItem
import GoalInput from './components/GoalInput';   // Import GoalInput

export default function App() {
  const [courseGoals, setCourseGoals] = useState([]);

  function addGoalHandler(enteredGoalText) {
    setCourseGoals((currentCourseGoals) => [
      ...currentCourseGoals,
      { text: enteredGoalText, key: Math.random().toString() },
    ]);
  }

  return (
    <View style={styles.appContainer}>
      <Image
        source={{ uri: 'https://www.softo.org/imgs/blog/2023/07/1690190032univer-png.png'
}}
        style={styles.logo}
      />

      {/* Pass the prop here */}
      <GoalInput onAddGoal={addGoalHandler} />
```

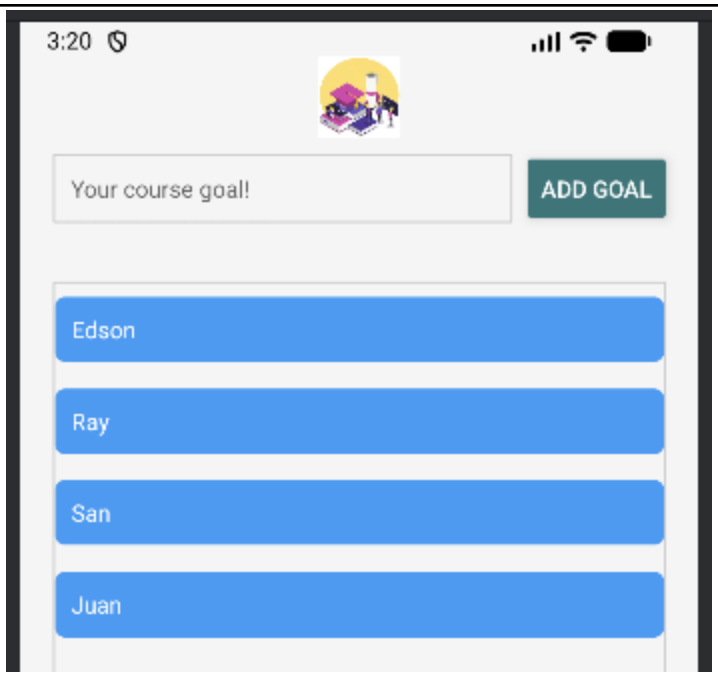
```
      <View style={styles.goalListContainer}>
        <FlatList
          data={courseGoals}
          renderItem={(itemData) => (
            <GoalItem text={itemData.item.text} />
          )}
        />
      </View>
    </View>
  );
}

const styles = StyleSheet.create({
  appContainer: {
    flex: 1,
    padding: 20,
    backgroundColor: '#f5f5f5',
  },
  logo: {
    width: 50,
    height: 50,
    marginBottom: 10,
    alignSelf: 'center',
  },
  goalListContainer: {
    flex: 4,
    marginTop: 16,
    borderWidth: 1,
    borderColor: '#ccc',
  },
});
```

7. Supplementary Activity

For this activity, you must demonstrate your ability to further break down your application into custom components.

- Show a screenshot of your application so far.




```
function addGoalHandler(enteredGoalText) {
  setCourseGoals((currentCourseGoals) => [
    ...currentCourseGoals,
    { text: enteredGoalText, key: Math.random().toString() },
  ]);
}
```

- This block of code manages the state of courseGoals and rendering list

ILO1: Utilize React Native components to make scrollviews and optimized lists.

- Limit the size of your goal list's container. Demonstrate the difference between scrollview and flatlist. Screenshots must be provided.
  - FlatList

```
<View style={styles.goalListContainer}>
  <FlatList
    data={courseGoals}
    renderItem={(itemData) => (
      <GoalItem text={itemData.item.text} />
    )}
  />
</View>
```

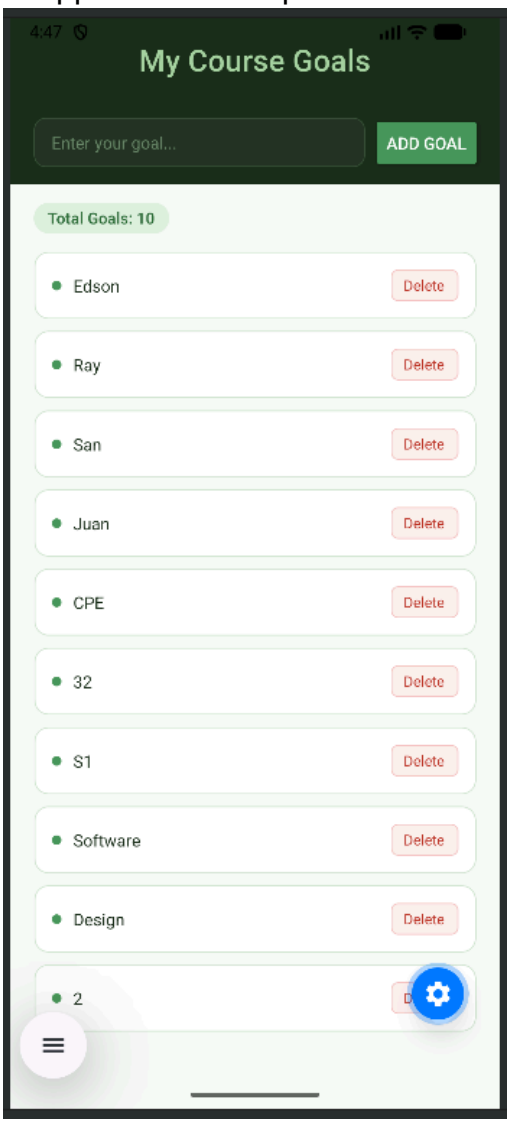
  -
  - Only items visible on screen are rendered. As you scroll, React Native dynamically loads/unloads items, making it much more efficient for large lists.
  - ScrollView

```
<View style={styles.goalListContainer}>
  <ScrollView>
    {courseGoals.map((goal) => (
      <GoalItem key={goal.key} text={goal.text} />
    ))}
  </ScrollView>
</View>
```

- Renders all items at once. If you add 20+ goals, performance slows because every item is kept in memory.

ILO2: Decompose app component into multiple custom components.

- Duplicate the activity by creating your own custom components to reflect how you further customized the application as required in the first activity.



- In this ILO2, the application was successfully decomposed into multiple custom components such as Header, GoalInput, GoalList, and GoalItem.

Each component was designed to handle a specific part of the interface, making the code more organized, reusable, and easier to maintain. Additional features like a delete functionality was added, allowing users to remove goals by tapping the item, with the delete option positioned on the right side for better user interface design. A dynamic counter was also introduced to display the total number of goals in real time.

JavaScript

```javascript
// App.js
import { useState } from 'react';
import { StyleSheet, View, Image, Text } from 'react-native';
import GoalInput from './components/GoalInput';
import GoalList from './components/GoalList';
import Header from './components/Header';

export default function App() {
  const [courseGoals, setCourseGoals] = useState([]);

  function addGoalHandler(enteredGoalText) {
    if (enteredGoalText.trim().length === 0) return;
    setCourseGoals((currentGoals) => [
      ...currentGoals,
      { text: enteredGoalText, key: Math.random().toString() },
    ]);
  }

  function deleteGoalHandler(id) {
    setCourseGoals((currentGoals) =>
      currentGoals.filter((goal) => goal.key !== id)
    );
  }

  return (
    <View style={styles.appContainer}>
      <Header />
      <GoalInput onAddGoal={addGoalHandler} />
      <Text style={styles.counter}>Total Goals: {courseGoals.length}</Text>
      <GoalList goals={courseGoals} onDeleteGoal={deleteGoalHandler} />
    </View>
  );
}

const styles = StyleSheet.create({
  appContainer: {
    flex: 1,
    backgroundColor: '#f7faf7',
  },
  logo: {
    width: 60,
    height: 60,
```

```
    marginBottom: 15,
    alignSelf: 'center',
  },
  counter: {
    fontSize: 13,
    fontWeight: '500',
    color: '#2d6e3a',
    backgroundColor: '#dff0df',
    alignSelf: 'flex-start',
    marginHorizontal: 20,
    marginBottom: 10,
    paddingHorizontal: 12,
    paddingVertical: 4,
    borderRadius: 20,
    overflow: 'hidden',
  },
});

// The main component that manages the app’s state, handles adding and deleting goals, and
connects all custom components together.
```

JavaScript

```
// Header.js
import { View, Text, StyleSheet } from 'react-native';

function Header() {
  return (
    <View style={styles.header}>
      <Text style={styles.title}>My Course Goals</Text>
    </View>
  );
}

const styles = StyleSheet.create({
  header: {
    backgroundColor: '#1a2e1a',
    paddingVertical: 18,
    paddingHorizontal: 20,

  },
  title: {
    color: '#a8d5a2',
    fontSize: 25,
    fontWeight: '500',
    textAlign: 'center',
    letterSpacing: 0.3,
  },
});
```

```
export default Header;

// Displays the title or header of the application to provide a clear identity and
structure for the UI.
```

JavaScript

```
// GoalItem.js
import { View, Text, StyleSheet, Pressable } from 'react-native';

function GoalItem(props) {
  return (
    <Pressable onPress={() => props.onDelete(props.id)}>
      <View style={styles.goalItem}>
        <View style={styles.leftRow}>
          <View style={styles.dot} />
          <Text style={styles.goalText}>{props.text}</Text>
        </View>
        <Text style={styles.deleteText}>Delete</Text>
      </View>
    </Pressable>
  );
}

const styles = StyleSheet.create({
  goalItem: {
    backgroundColor: '#ffffff',
    borderWidth: 1,
    borderColor: '#d4e8d4',
    borderRadius: 12,
    padding: 14,
    marginVertical: 5,
    flexDirection: 'row',
    justifyContent: 'space-between',
    alignItems: 'center',
  },
  leftRow: {
    flexDirection: 'row',
    alignItems: 'center',
    gap: 10,
  },
  dot: {
    width: 8,
    height: 8,
    borderRadius: 4,
    backgroundColor: '#4a9a5a',
  },
```

```
  goalText: {
    color: '#1a2e1a',
    fontSize: 14,
  },
  deleteText: {
    fontSize: 12,
    color: '#c0392b',
    backgroundColor: '#fdf0ef',
    borderWidth: 1,
    borderColor: '#f5c6c3',
    borderRadius: 6,
    paddingHorizontal: 10,
    paddingVertical: 4,
    overflow: 'hidden',
  },
});

export default GoalItem;

// Represents each individual goal, showing its text and allowing users to delete it by
tapping the item.
```

JavaScript

```
// GoalList.js
import { View, FlatList, StyleSheet } from 'react-native';
import GoalItem from './GoalItem';

function GoalList(props) {
  return (
    <View style={styles.goalListContainer}>
      <FlatList
        data={props.goals}
        renderItem={(itemData) => (
          <GoalItem
            text={itemData.item.text}
            id={itemData.item.key}
            onDelete={props.onDeleteGoal}
          />
        )}
      />
    </View>
  );
}

const styles = StyleSheet.create({
  goalListContainer: {
    flex: 1,
```

```
    paddingHorizontal: 20,
  },
});

export default GoalList;

// Responsible for displaying the list of goals using a FlatList and passing data to each
goal item.
```

JavaScript

```
// GoalInput.js
import { useState } from 'react';
import { View, TextInput, Button, StyleSheet } from 'react-native';

function GoalInput(props) {
  const [enteredGoalText, setEnteredText] = useState('');

  function textInputHandler(text) {
    setEnteredText(text);
  }

  function addGoalHandler() {
    props.onAddGoal(enteredGoalText);
    setEnteredText('');
  }

  return (
    <View style={styles.inputContainer}>
      <TextInput
        placeholder="Enter your goal..."
        placeholderTextColor="#6a8a6a"
        style={styles.textInput}
        onChangeText={textInputHandler}
        value={enteredGoalText}
      />
      <Button title="ADD GOAL" onPress={addGoalHandler} color="#4a9a5a" />
    </View>
  );
}

const styles = StyleSheet.create({
  inputContainer: {
    flexDirection: 'row',
    alignItems: 'center',
    backgroundColor: '#1a2e1a',
    paddingHorizontal: 20,
    paddingVertical: 14,
    marginBottom: 16,
```

```
    gap: 10,
  },
  textInput: {
    flex: 1,
    backgroundColor: '#243324',
    borderWidth: 1,
    borderColor: '#3a533a',
    borderRadius: 10,
    paddingHorizontal: 14,
    paddingVertical: 10,
    color: '#a8d5a2',
    fontSize: 14,
  },
});

export default GoalInput;

// Handles user input by allowing users to type and add new goals, then passes the data to
the main component.
```

8. Conclusion

- **A concise summary of the activity performed.**

In this activity, I created a simple React Native application based from the previous activity 10.2 by experimenting with ScrollView and FlatList to display a growing list of goals. I observed how ScrollView renders all items at once, while FlatList optimizes performance by rendering only visible items. I then decomposed the app into multiple custom components GoalInput, GoalItem, and GoalList to make the code cleaner, modular, and easier to style. I also customized the design to make the interface more visually appealing.

- **Your personal conclusions / reflections.**

I concluded in this activity that I did was break down the app into smaller components made the project easier to manage and understand. It allowed me to isolate functionality and styling, which improved readability and reusability. I also realized that while ScrollView is simple, FlatList is the better choice for larger datasets because of its efficiency. Customizing the design gave me confidence in making apps not only functional but also user-friendly and attractive.

- **An overview of the lessons you learned from this module.**
  - The difference between ScrollView and FlatList, and when to use each.
  - How to decompose an app into multiple components for better organization.
  - The importance of props in passing data between components.
  - How styling can transform a basic app into something more engaging and professional.

9. Assessment Rubric

T.I.P. SO 7

Criteria	Ratings						Pts
T.I.P. SO 7.1 Acquire and apply new knowledge from outside sources threshold: 4.2 pts	6 pts [Excellent] Educational interests and pursuits exist and flourish outside classroom requirements,knowledge and/or experiences are pursued independently and applies knowledge learned into practice	5 pts [Good] Educational interests and pursuits exist and flourish outside classroom requirements,knowledge and/or experiences are pursued independently	4 pts [Satisfactory] Look beyond classroom requirements, showing interest in pursuing knowledge independently	3 pts [Unsatisfactory] Begins to look beyond classroom requirements, showing interest in pursuing knowledge independently	2 pts [Poor] Relies on classroom instruction only	1 pts [Very Poor] No initiative or interest in acquiring new knowledge	6 pt
T.I.P. SO 7.2 Learn independently threshold: 4.2 pts	6 pts [Excellent] Completes an assigned task independently and practices continuous improvement	5 pts [Good] Completes an assigned task without supervision or guidance	4 pts [Satisfactory] Requires minimal guidance to complete an assigned task	3 pts [Unsatisfactory] Requires detailed or step-by-step instructions to complete a task	2 pts [Poor] Shows little interest to complete a task independently	1 pts [Very Poor] No interest to complete a task independently	6 pt
T.I.P. SO 7.3 Critical thinking in the broadest context of technological change threshold: 4.2 pts	6 pts [Excellent] Synthesizes and integrates information from a variety of sources; formulates a clear and precise perspective; draws appropriate conclusions	5 pts [Good] Evaluate information from a variety of sources; formulates a clear and precise perspective.	4 pts [Satisfactory] Analyze information from a variety of sources; formulates a clear and precise perspective.	3 pts [Unsatisfactory] Apply the gathered information to formulate the problem	2 pts [Poor] Gather and summarized the information from a variety of sources but failed to formulate the problem	1 pts [Very Poor] Gather information from a variety of sources	6 pt
T.I.P. SO 7.4 Creativity and adaptability to new and emerging technologies threshold: 4.2 pts	6 pts [Excellent] Ideas are combined in original and creative ways in line with the new and emerging technology trends to solve a problem or address an issue.	5 pts [Good] Ideas are creative and adapt the new knowledge to solve a problem or address an issue	4 pts [Satisfactory] Ideas are creative in solving a problem, or address an issue	3 pts [Unsatisfactory] Shows some creative ways to solve the problem	2 pts [Poor] Shows initiative and attempt to develop creative ideas to solve the problem	1 pts [Very Poor] Ideas are copied or restated from the sources consulted	6 pt

Total Points: 24

10. Additional Screenshots